

Go 1.25 — стиль кода my-chat

Проект на **Go 1.25** ([go.mod](#)). При написании и ревью Go-кода предпочитай идиомы ниже вместо устаревших паттернов «как в 1.21 и раньше».

Типы и ошибки

```
// ❌  
var v interface{}  
err = fmt.Errorf("load: %v", err)  
  
// ✅  
var v any  
err = fmt.Errorf("load: %w", err) // сохраняй цепочку; несколько —  
errors.Join
```

Встроенные и stdlib вместо самописного

Вместо (старый стиль)	Использовать
ручной min/max	<code>min(a, b), max(a, b)</code>
цикл обнуления map/slice	<code>clear(m), clear(s)</code>
<code>if x == nil *x == ""</code> цепочки дефолтов	<code>cmp.Or(a, b, c)</code>
<code>sync.Once</code> + замыкание	<code>sync.OnceFunc</code> / <code>sync.OnceValue</code> / <code>OnceValues</code>
копирование строки через <code>string([]byte(...))</code> с aliasing	<code>strings.Clone</code> / <code>bytes.Clone</code>
самописные helpers для slice/map	пакеты <code>slices</code> , <code>maps</code>

```
// ❌  
for i := 0; i < n; i++ { ... }  
limit := a; if b < a { limit = b }  
  
// ✅  
for i := range n { ... } // Go 1.22+  
limit := min(a, b)  
name := cmp.Or(cfg.Name, "default")
```

Циклы и итерации

- `for i := range n` для простых счётчиков (1.22+).

- Не полагайся на «общее» значение переменной цикла между итерациями — семантика `per-iteration` уже стандарт (1.22+); не копируй `x := x` без нужды.
- Для обхода/фильтрации/поиска по `slice` — `slices.*` (`Contains`, `Clone`, `Sorted`, `DeleteFunc`, ...), не изобретай заново.

Context

```
// ❌ только Cancel без причины при бизнес-ошибке
ctx, cancel := context.WithCancel(parent)

// ✅ когда важна причина отмены (1.20+)
ctx, cancel := context.WithCancelCause(parent)
cancel(fmt.Errorf("worker stopped: %w", err))
// читать: context.Cause(ctx)
```

Аналогично `WithTimeoutCause` / `WithDeadlineCause`, если задаёте дедлайн с причиной.

Чего не делать

- Не откатывайся к `interface{}`, ручным `min/max` и «классическому» `sync.Once`, если есть стандартный эквивалент.
- Не тяни сторонние `libs` ради того, что уже есть в `slices` / `maps` / `cmp` / `sync`.
- Не предлагай API/синтаксис новее, чем **1.25**, и не требуй фич «из будущего».

Качество после правок: `task lint`, при необходимости `task test` / `task test:integration` (см. rule линтера).